Context Engineering, skills & workflows

Du prompt improvisé au harness maîtrisé

5 Degrés · 15 et 16 octobre 2026 · Anthony Denneulin

Émargement de jeudi matin : signez en arrivant

`edusign.app/sc/61HT2NA3`

Qui suis-je ?

- **CTO et cofondateur de LiveMeUp** : SaaS de vidéo commerce (live shopping, vidéo shoppable), utilisé par plus de 1 400 entreprises dans 35 pays
- **Plus de 15 ans** en architecture SaaS et automatisation : Node.js/TypeScript, architectures événementielles, AWS, Azure
- **Avant** : Cegid (architecte solution), Tokeny (lead dev), KPMG (lead dev, Scrum Master), ConsenSys
- **E-mail** : a.denneulin@gmail.com
- **LinkedIn** : linkedin.com/in/anthonydenneulin

Le plan de ce support a été préparé avec Claude Code.

Le cadre

Sujet	Concrètement
Rythme	théorie courte → démo → pratique ; jamais plus de 30 min de théorie d'affilée
Terrain	vos repos ; les démos sur un vrai projet open source
Questions	à tout moment ; celles qui nous emmèneraient trop loin vont au tableau
Binômes	je propose de mélanger les profils ; sinon, par affinités : c'est vous qui voyez
Entre nous	vous ou tu : à vous de choisir

Les deux jours

J1 matin

Comprendre & configurer

fenêtre de contexte · CLAUDE.md

J1 après-midi

Hooks & skills

automatismes · capacités

J2 matin

Composer & piloter la boucle

workflows spec-driven · loops

J2 après-midi

Scaler par la parallélisation

worktrees · sous-agents

Du prompt improvisé au harness maîtrisé

Ce que vous emportez

J1 matin

1. Un CLAUDE.md audité et optimisé, complété de règles `.claude/rules/`

J1 après-midi

2. Un hook déterministe et un skill personnalisé, versionnés dans votre repo

J2 matin

3. Un plan de composition de skills sur une tâche réelle de votre backlog

J2 après-midi

4. Un workflow multi-agent testé avec worktrees et sous-agents

dans le livret

5. Une checklist des pratiques de context engineering à adopter

fil des deux jours

6. Un plan d'adoption à 30 jours pour votre équipe



Le livret · 1 à 4 produits en atelier ou en exercice, 5 et 6 dans le livret

`denneulin.github.io/livret-context-engineering`

le cahier, à télécharger maintenant, hors de votre repo · le récap, vendredi à 16h30

Tour de table express

Votre prénom

et votre stack

Votre usage de l'IA

et votre setup aujourd'hui

Votre attente principale

et une frustration avec l'IA aujourd'hui

1 minute chacun · attentes et frustrations vont au tableau.

Ce que vous nous avez dit

5/6
ont au moins 5 ans d'expérience

3/6
utilisent l'IA de façon occasionnelle

0/6
écrivent les tests avant le code

4/6
politique IA en cours de définition

Vos attentes, à parts égales

- comprendre l'agentic coding **J1**
- gagner en productivité **J1–J2**
- MCP, sous-agents, orchestration **J1 après-midi + J2**

Questionnaire de positionnement : 6 réponses sur 9 inscrits

Panorama

De l'assistant à l'agent

Assistant, agent, orchestrateur



Copilot (complétion) · ChatGPT

Claude Code · Cursor

demain, J2

Ce qui change d'une étape à l'autre : ce que l'outil peut faire seul, et donc ce qu'il doit savoir.

Le paysage en quatre étages

Surfaces de pilotage
où vous lancez, suivez, validez
terminal · IDE · app desktop · web
ADE : Conductor, Orca…

Votre harness
ce que vous configurez : ces deux jours
CLAUDE.md · règles · mémoire · hooks
skills · sous-agents · MCP · permissions

Agents de code
la loop, les outils, les permissions
Claude Code · Codex · Copilot · Cursor
ouverts : Pi (minimal) · OpenCode

Modèles
génèrent le texte et les appels d'outils
Claude · GPT · Gemini
open-weight : DeepSeek · Kimi · GLM

Vibe coding
Lovable · Bolt
v0 · Replit

les quatre étages
dans un seul
produit web

rapide,
peu de contrôle

Un choix à chaque étage. Ces deux jours : l'étage doré, votre harness.

Source : schéma d'après Florian Bruniaux, « Agent Harness Map », cc.bruniaux.com, 2026

Adoption et confiance en 2026

90 %

des développeurs pros utilisent des agents de code au moins chaque semaine (JetBrains, mai-juillet 2026)

48 %

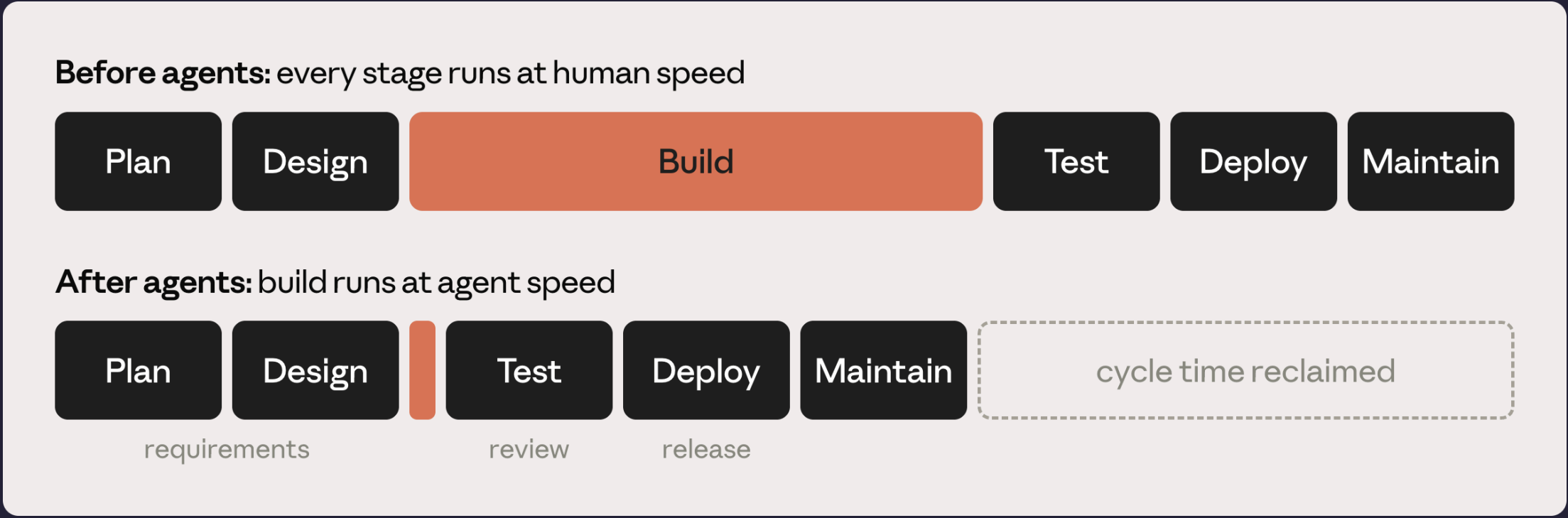
font confiance à l'IA quand la sortie est facile à vérifier (Stack Overflow 2026)

Ce qui prive du bon contexte au travail : info incomplète **63 %** · contexte resté dans les têtes **61 %** · outils IA qui en manquent **20 %** (Stack Overflow 2026)

La confiance vient de la vérification et du contexte.

Sources : JetBrains, résultats partiels 2026 · Stack Overflow Developer Survey 2026 (~31 000 réponses)

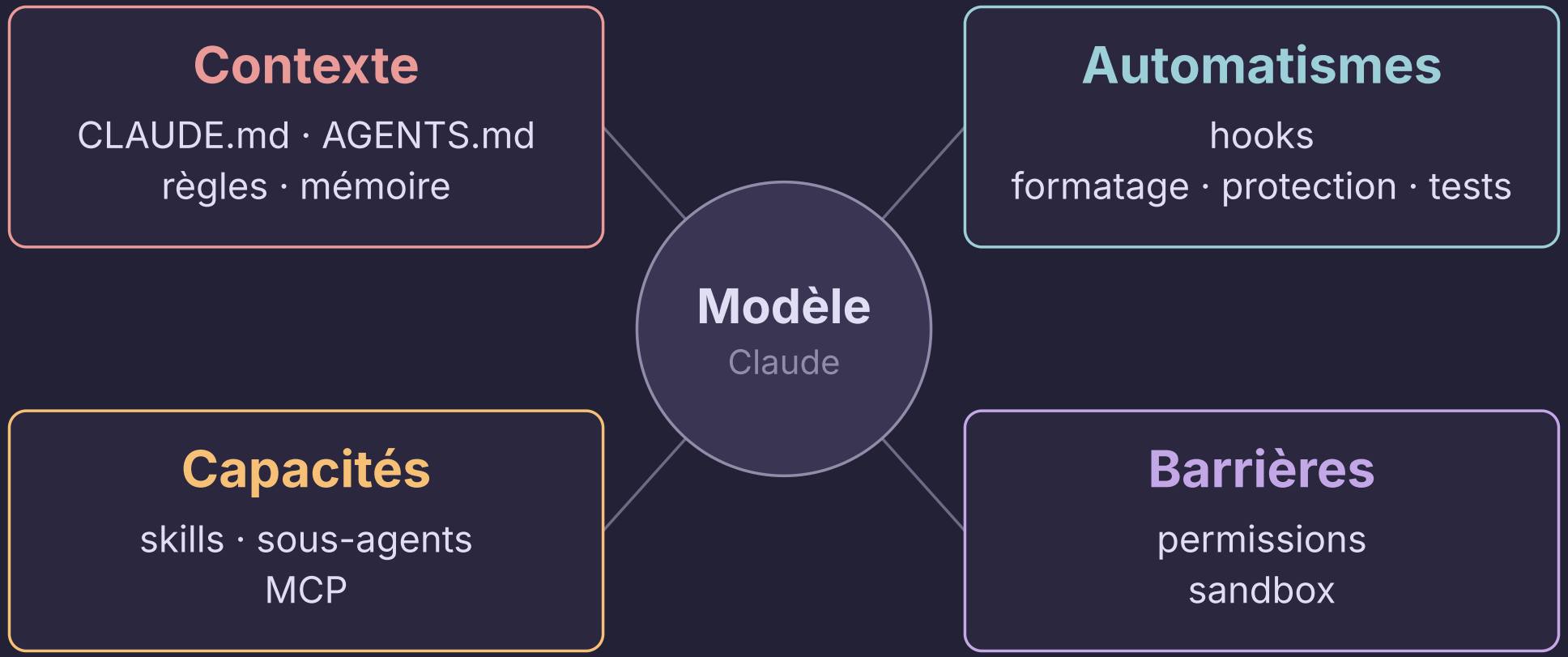
Le goulot s'est déplacé



L'agent écrit vite. Restent à vitesse humaine : dire quoi construire, et vérifier.

Source : Anthropic, « The AI-native SDLC playbook », 21/08/2026, figure 1

Ce n'est pas le modèle, c'est le harness

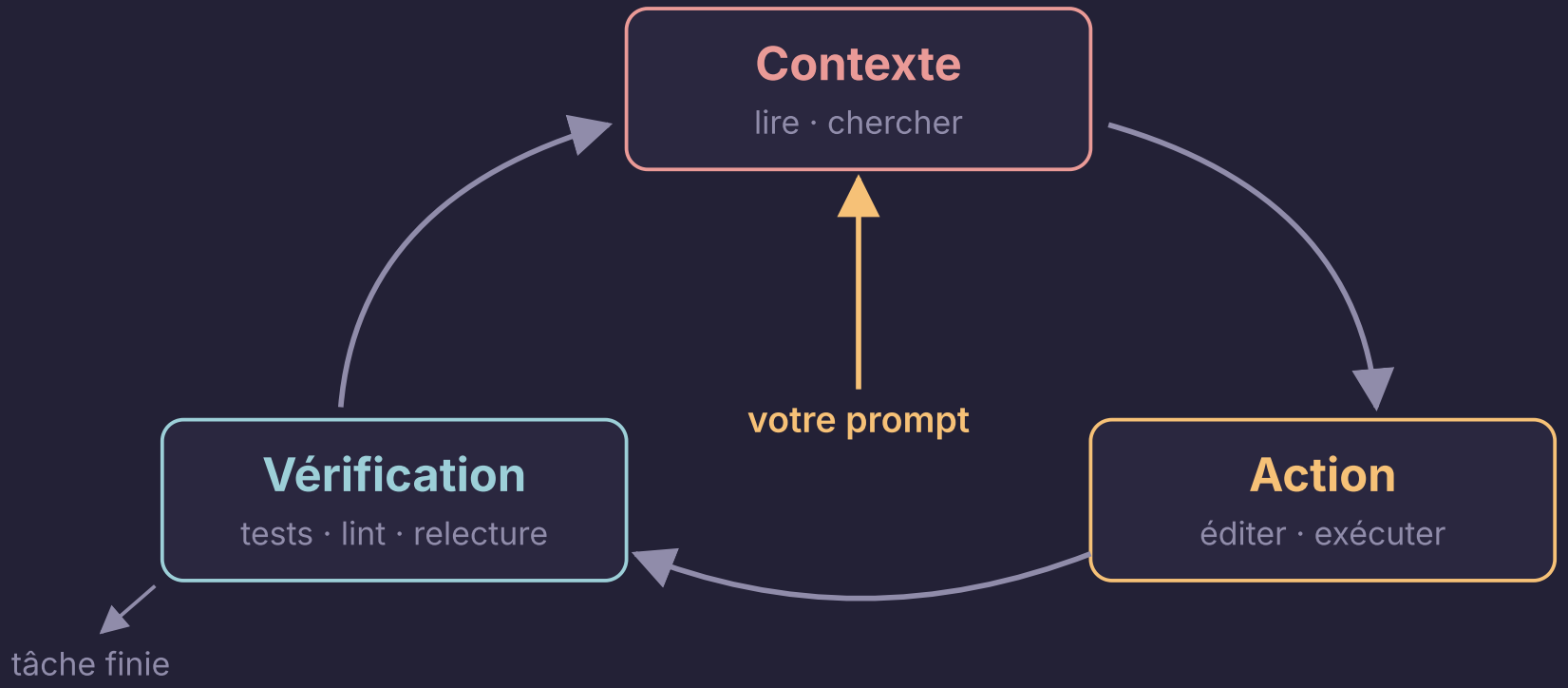


Le harness : tout ce que vous configurez autour du modèle.

Premiers pas avec un agent

Comment il travaille, et ce qu'il vous demande

La loop d'un agent



Un agent = un modèle + des outils + une loop.

Ce que Claude Code peut faire, et ce qu'il demande

Pour…	Outils
Lire, chercher	`Read` ; `grep`, `find` par `Bash` (`Grep`, `Glob` sous Windows)
Écrire	`Edit`, `Write`
Exécuter	`Bash`
Aller sur le web	`WebFetch`, `WebSearch`
Déléguer, se brancher	sous-agents, MCP

Permissions

Chaque action sensible attend votre accord : **oui**, **oui et ne plus demander** (souvent pour tout un motif, comme `npm test *`), ou **non**.

Le projet peut pré-autoriser des commandes (`permissions.allow`) ou en interdire (`permissions.deny`) dans `.claude/settings.json`.

Les modes et les raccourcis

Mode (Shift+Tab fait défiler)	Ce que fait Claude	Quand l'utiliser
Auto (mode de départ)	les actions jugées sûres passent sans demander	le quotidien, sur un repo bien configuré
Manual	demande avant les actions sensibles	quand vous voulez tout valider
Accept edits	édite les fichiers sans demander	plan validé, vous relirez le diff
Plan	propose un plan, ne modifie pas vos fichiers	première fois sur un repo, tâche floue
Bypass permissions	`claude --dangerously-skip-permissions` : tout passe sans demander	seulement dans un conteneur ou une VM isolés

Esc
interrompre

Esc Esc
revenir en arrière

@fichier
référencer un fichier

/plan ...
entrer en Plan mode

DÉMO Premier contact avec Bénéfriches

Bénéfriches : application open source de l'ADEME (beta.gouv), monorepo TypeScript : API NestJS, front React, tests e2e Playwright, package partagé

1. **Plan mode** · « Explique-moi en dix lignes à quoi sert ce projet et comment il est organisé. »
2. **Plan mode** · « Quelles commandes lancent les tests du package shared ? »
3. **Sortie du Plan mode** · « Lance les tests du package shared et résume le résultat. »

Regardez ce que Claude lit, et ce qu'il ne vous demande pas.

EXERCICE Premier contact avec votre repo

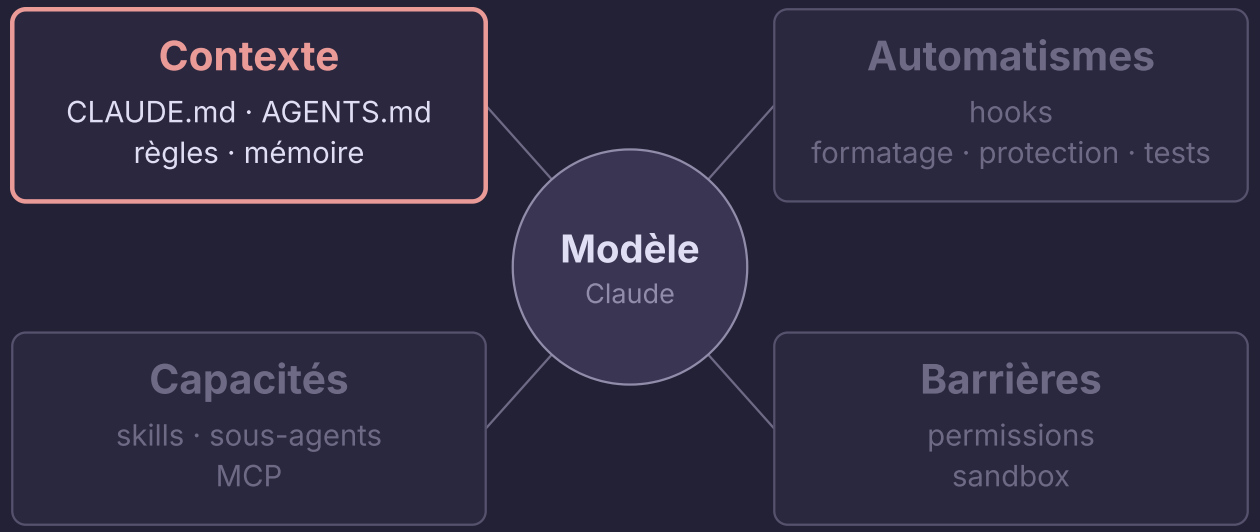
1. Lancez `claude` à la racine de votre repo
2. Passez en **Plan mode** (Shift+Tab)
3. Demandez : « Explique l'architecture de ce projet et liste les commandes pour builder, tester et linter. »
4. Notez dans le livret ce qui est **juste**, **faux** ou **manquant**

Vos notes sont la matière de l'Atelier 1, à midi.

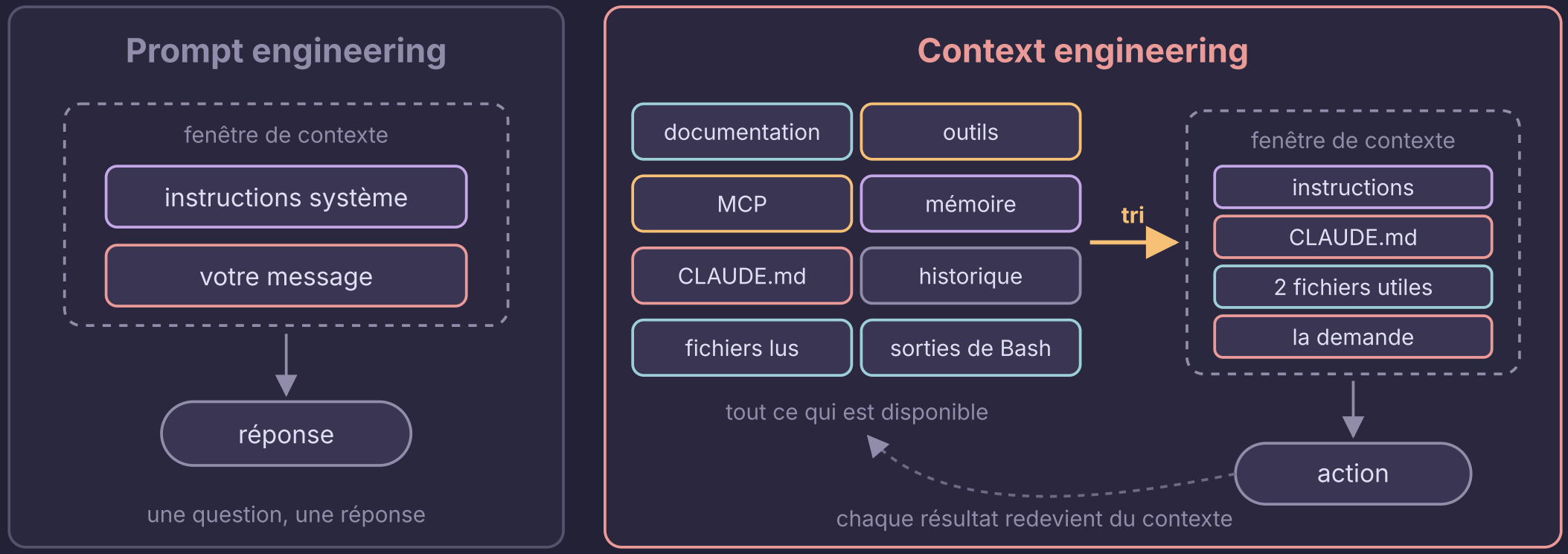
10 minutes · seul ou à deux sur un poste

Le context engineering

Ce que l'agent voit, c'est ce qu'il produit



Du prompt au contexte



Andrej Karpathy (juin 2025, paraphrase) : l'art de remplir la fenêtre de contexte avec exactement l'information utile à l'étape suivante.

Anthropic (septembre 2025, paraphrase) : chercher le plus petit ensemble de tokens à forte valeur qui maximise la probabilité du résultat voulu.

Ce qui remplit réellement la fenêtre



- **1** Instructions système et outils intégrés principaux
- **2** Outils MCP : leurs noms (définitions à la demande)
- **3** Fichiers mémoire : CLAUDE.md, AGENTS.md, règles
- **4** Liste des skills (nom + description)
- **5** Historique : vos prompts, ses réponses
- **6** Résultats d'outils : fichiers lus, sorties de commandes

Chaque serveur MCP branché ajoute ses noms d'outils et ses instructions. Bénéfriches en a un : context7.

Répartition indicative en fin de session : regardez la vôtre avec `/context`

À vous : `/context`

1. Tapez

`/context` dans la session de l'exercice de tout à l'heure

2. Regardez

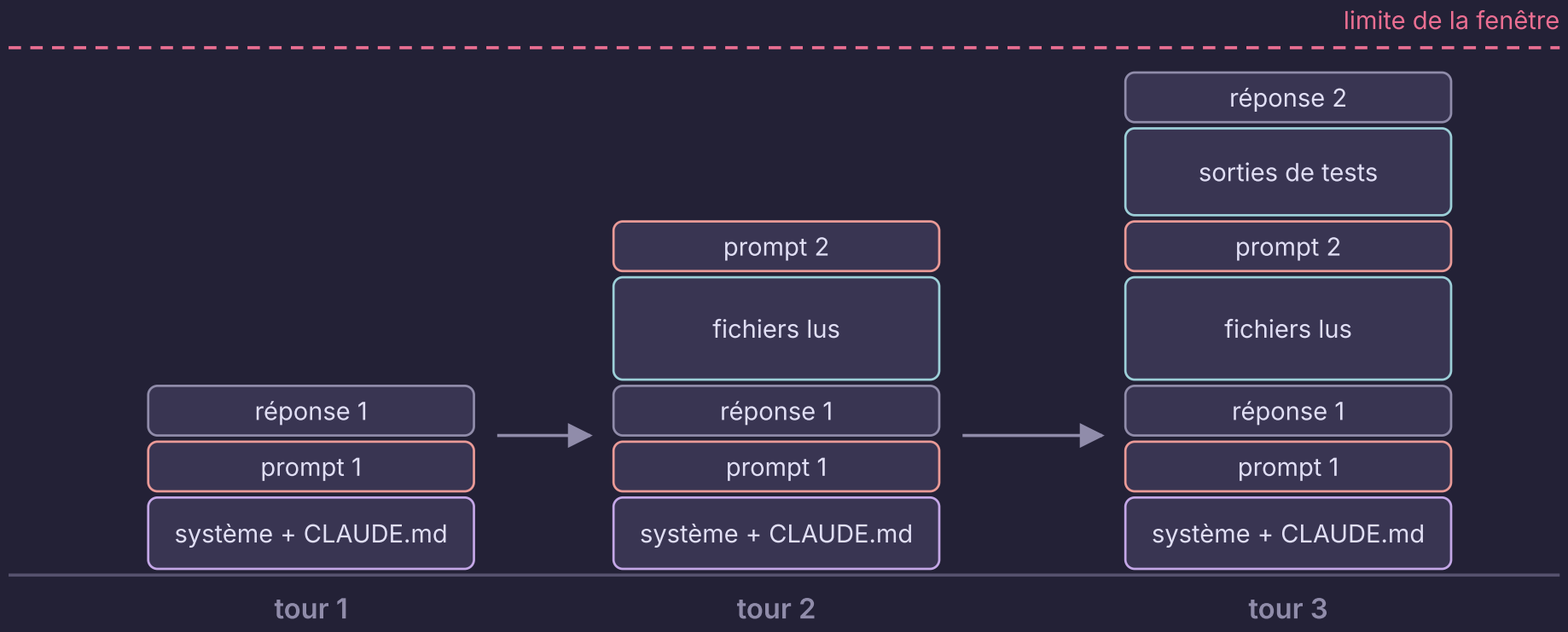
Quel poste prend le plus de place ? Lequel vous surprend ?

Plus de contexte n'est pas mieux. On voit pourquoi après la pause.

Pause

Reprise dans 15 minutes

Tout s'accumule



Le modèle relit tout, à chaque tour.

Plus de contexte ≠ meilleure réponse

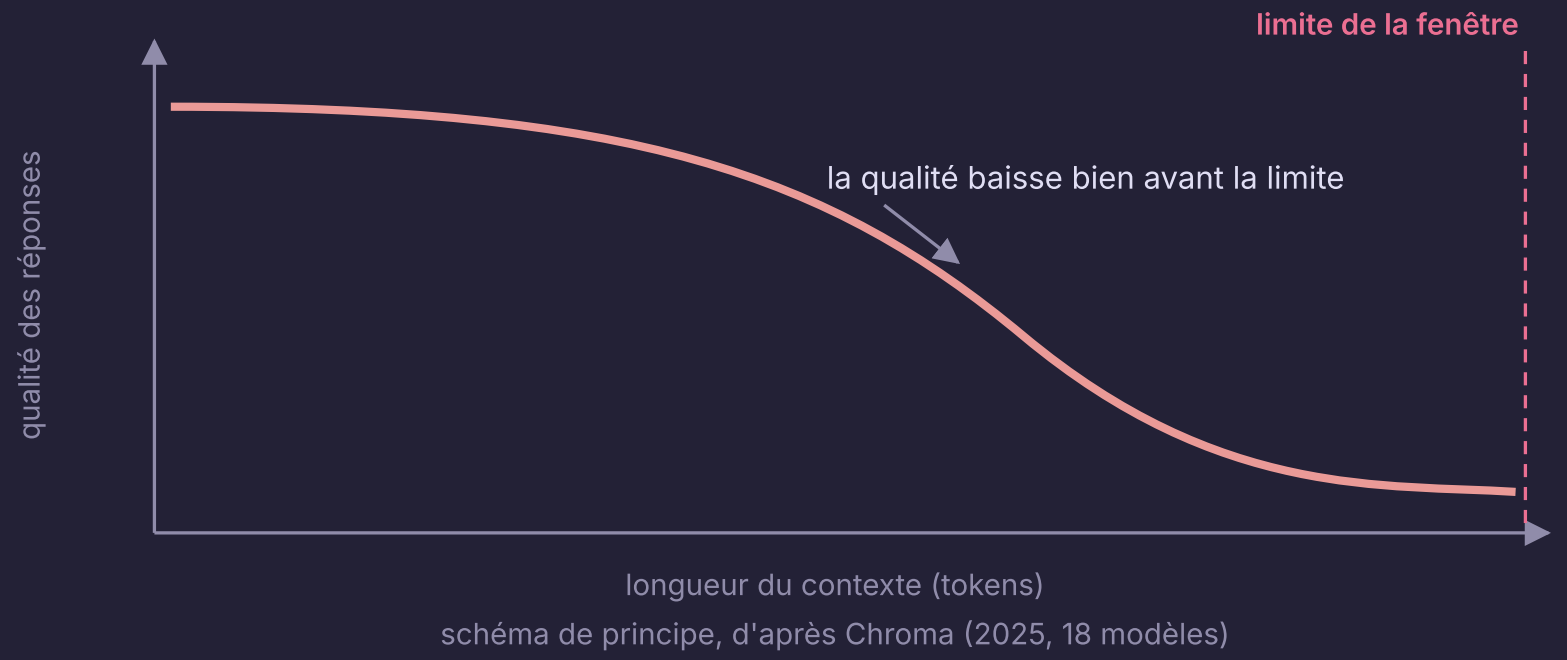


schéma de principe, d'après Chroma (2025, 18 modèles)

Context rot · Chroma, 2025, 18 modèles dont Claude : la performance baisse quand l'entrée s'allonge, même sur des tâches simples.

Budget d'attention · Anthropic, 2025 : le modèle a un budget d'attention limité ; chaque token ajouté en consomme une part.

Une fenêtre de 1M de tokens n'est pas une invitation à la remplir.

Lost in the middle, version 2026

Ce qu'on disait en 2023

Liu et al. : une information placée **au milieu** d'un long contexte est moins bien exploitée qu'au début ou à la fin.

Ce qu'on sait en 2026

Effet **contesté** sur les modèles récents : Chroma ne le retrouve pas sur sa tâche « aiguille dans une botte de foin » ; Veseli et al. (2025) le voient surtout quand l'entrée occupe moins de la moitié de la fenêtre.

La longueur reste le vrai problème.

Le conseil qui reste vrai : les documents longs en tête, la question à la fin.

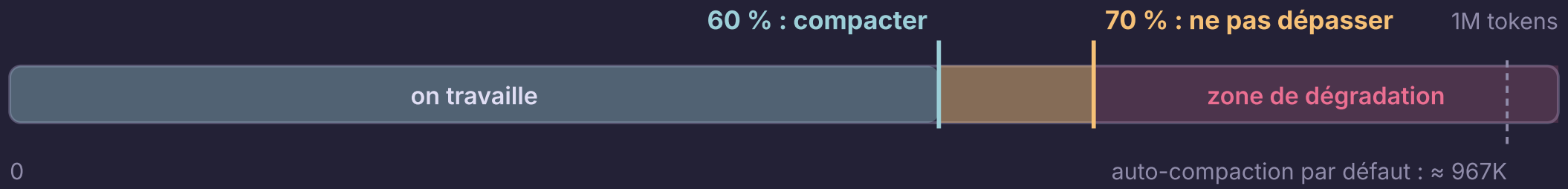
Liu et al., 2023-2024 · Chroma, « Context Rot », 2025 · Veseli et al., 2025 · documentation Claude

Les 4 modes de défaillance

Mode	Ce qui se passe	Exemple de dev
Poisoning (empoisonnement)	une erreur entre dans le contexte et y est réutilisée	une API inventée par Claude, puis réutilisée dans toute la session
Distraction	le contexte est si long que le modèle s'y accroche et néglige ce qu'il a appris	une session de trois heures où Claude répète ses actions passées
Confusion	du contenu superflu dégrade la réponse	trente outils MCP chargés pour une tâche qui en utilise deux
Clash (conflit)	une information ajoutée contredit ce qui est déjà là	le CLAUDE.md dit Jest, un message plus loin dit Vitest

Drew Breunig, « How Long Contexts Fail », 22 juin 2025

La règle des 70 %, et les commandes



La règle

Une **heuristique**, pas un seuil documenté. Par défaut, Claude Code vous laisse aller jusqu'à 97 %.

`/autocompact <taille|auto>` permet d'appliquer la règle.

Situation	Commande
Tâche terminée	`/clear`
Même tâche, contexte lourd	`/compact <ce qu'il faut garder>`
Mauvaise direction	Esc Esc (`/rewind`)
Savoir où on en est	`/context`

DÉMO Reprendre la main sur le contexte

1. `/clear` puis `/context` : la fenêtre au départ
2. Faire lire tout un dossier, puis `/context` : elle a grossi
3. `/compact <ce qu'il faut garder>`, puis `/context` : elle a fondu
4. Accept edits, un renommage, Esc, puis Esc Esc : retour à l'état d'avant
5. `/clear` : nouvelle tâche, contexte propre

Regardez les chiffres de `/context` entre chaque étape.

Quatre verbes pour un prompt précis

Référencer

`@src/app/orders/order-list.component.ts`

un chemin, un symbole, l'erreur exacte

Montrer

« sur le modèle de `order-list.component.ts` »

un exemple, un fichier modèle

Contraindre

« ne touche pas à `ProductService` »,
« `npm test` doit passer »

une limite, le critère de vérification

Structurer

« propose-moi d'abord un plan »

des étapes, un format de sortie

Chaque verbe retire une chose que l'agent aurait dû deviner, ou chercher.

Avant / après (Angular)

Avant

« Ajoute un filtre à la liste des produits. »

Après

« Dans `@src/app/products/product-list.component.ts`, ajoute un filtre par catégorie sur le modèle de `order-list.component.ts` (même pattern signal + computed). Ne touche pas à `ProductService`. Ajoute un test dans `product-list.component.spec.ts` ; `npm test -- product-list` doit passer. Propose-moi d'abord un plan. »

Où sont les quatre verbes ?

Avant / après (.NET)

Avant

« Corrige le bug des commandes. »

Après

« `OrdersController.Create` (`src/Api/Controllers/OrdersController.cs`) renvoie 500 quand `ShippingAddress` est null. Écris d'abord un test xUnit qui reproduit le bug dans `OrdersControllerTests`, puis corrige. `dotnet test --filter OrdersController` doit passer. »

Le symptôme exact, le test qui le reproduit, la commande qui prouve la correction.

Contraindre, c'est donner un critère de vérification

Sans critère

prompt → code → « C'est fait. »

L'agent s'arrête quand il **pense** avoir fini.

Avec critère

prompt → code → test rouge → correction → test vert → « C'est fait, le test passe. »

Il **vérifie** lui-même, et recommence si ça échoue.

Pas de tests ? Demandez-lui d'écrire le test d'abord.



EXERCICE

Réécrivez un prompt

Prenez un prompt flou que vous avez tapé cette semaine.
Pas d'idée ? Prenez celui-ci : **« Optimise la page de recherche. »**

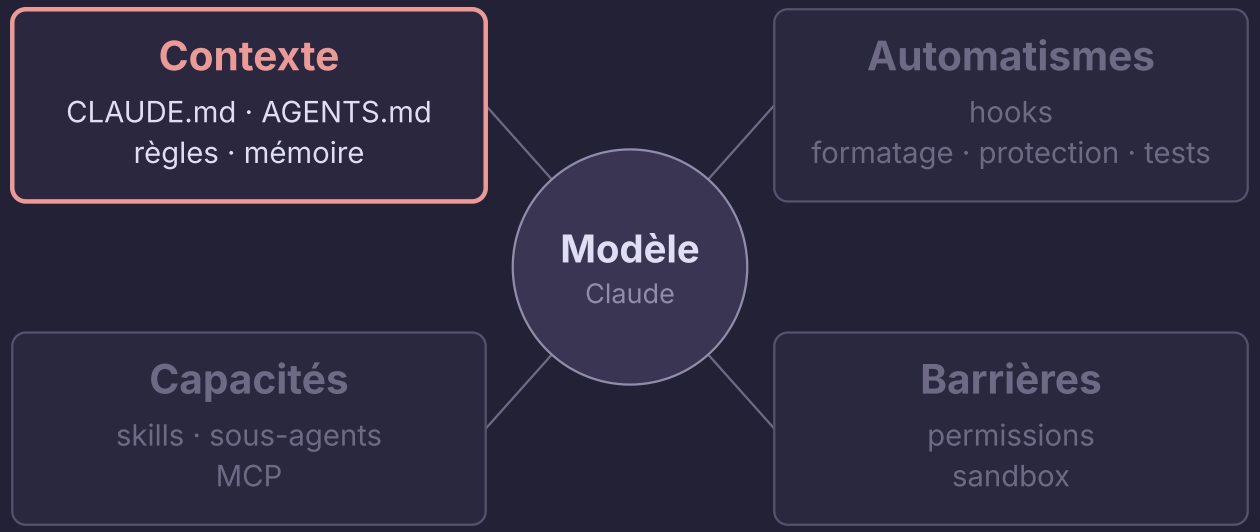
Réécrivez-le dans le livret avec les quatre verbes :

Référencer	**Montrer**	**Contraindre**	**Structurer**

3 minutes d'écriture · 1 minute pour un ou deux volontaires

CLAUDE.md, le README des agents

Ce qui doit être vrai à chaque session



Un vrai exemple : l'AGENTS.md de Bénéfriches

```
# Benefriches
Bénéfriches (ADEME incubator, beta.gouv) helps …
## Workspaces
pnpm monorepo; each workspace has its own AGENTS.md:
## Rules and nested instructions
## Conventions
- Run scripts with pnpm, not npm: `pnpm --filter …`
## Changing `packages/shared`
## Testing
## Git
```

87 lignes

identité du projet · workspaces · index des règles · conventions · quel test lancer · git

+ 5

un AGENTS.md par workspace (api, web, e2e-tests, shared), et un dans un dossier du front

Chargé à chaque session. Sans CLAUDE.md, Claude Code lit AGENTS.md.

Les 6 zones clés

Section du CLAUDE.md	La question à se poser	Exemple Angular / .NET
Commands	quelles commandes un nouveau ne devinerait pas ?	`npm run test:ci` · `dotnet test --filter Category=Unit`
Testing	quel framework, quelles conventions ?	Jest et Testing Library · xUnit et FluentAssertions
Pièges & environnement	qu'est-ce qui surprend un nouveau ?	l'API doit tourner avant `ng serve` `dotnet ef` exige `--startup-project`
Code style	quels patterns propres au projet ?	« suis `src/app/orders/` »
Git workflow	qu'est-ce qui n'est pas standard ?	conventional commits · squash
Boundaries	qu'est-ce qu'on ne touche jamais ?	`appsettings.Production.json` · `.env*` · migrations appliquées

Ce qu'on n'y met pas

À éviter

- ce que Claude déduit du code (« c'est une application Angular »)
- la description fichier par fichier
- les consignes vagues : « écris du code propre »

À faire

- ce que Claude **ne peut pas deviner**
- court (**moins de 200 lignes**) et précis : une commande, un chemin, un exemple

Un principe plutôt qu'une règle rigide :

Avant « Par défaut, n'écris aucun commentaire. Jamais de docstring de plusieurs paragraphes, ni de bloc de commentaires de plusieurs lignes : une ligne courte au plus. »

Après « Écris du code qui se lit comme le code qui l'entoure : même densité de commentaires, même nommage, mêmes idiomes. »

Source : Anthropic, « The new rules of context engineering for Claude 5 generation models », 24/07/2026 (traduit)

Votre code est le premier contexte

ETH Zurich, 2026 : un fichier de contexte augmente le coût d'inférence de **plus de 20 %** en moyenne, sans meilleur taux de réussite en général. Les consignes explicites sont suivies ; les **vues d'ensemble du dépôt n'aident pas**.

Ce que Claude lit	Chez vous
Des noms explicites	`calculateShippingFee()` plutôt que `calc()`
Une structure prévisible	une feature Angular par dossier, des controllers .NET à leur place
Des types stricts	TypeScript en `strict`, `<Nullable>enable</Nullable>` en C#
Des tests	des exemples d'usage, et une commande qui vérifie

Code lisible : CLAUDE.md court. Code confus : le CLAUDE.md compense, et vieillit.

Source : Gloaguen et al. (ETH Zurich, LogicStar), « Evaluating AGENTS.md », arXiv 2602.11988, v3 du 29/09/2026

Le golden file

Avant : des règles écrites à la main

```
## Style
- Composants standalone, OnPush
- État local en signals, dérivés en computed
- Injection avec inject(), pas de constructeur
- Pas de logique dans les templates
- Controllers : DTO en entrée, jamais l'entité
- Erreurs renvoyées en ProblemDetails
```

Après : un modèle à suivre

```
## Style
- Nouvelle feature Angular :
  suis src/app/orders/
- Nouveau controller .NET :
  suis OrdersController
```

Zéro maintenance : Claude lit le modèle quand il en a besoin.

Hiérarchie, règles et mémoire

Les fichiers mémoire, du plus général au plus local



`.claude/rules/angular.md`

```
---
paths:
  - "src/app/**/*.ts"
---
# Angular
- Composants standalone, état en signals
- Nouvelle feature : suis src/app/orders/
- Un .spec.ts à côté de chaque composant
```

Sans `paths`, une règle est chargée au démarrage ; avec `paths`, seulement quand Claude lit ou modifie un fichier qui correspond.

Mémoire automatique : `MEMORY.md`, ses 200 premières lignes sont chargées au démarrage · `/memory` pour la voir et l'éditer

L'entretenir

`/doctor prompt-audit`

Cherche les instructions **périmées**, les fichiers ou commandes **qui n'existent plus**, les fichiers **qui se contredisent**.

`/doctor`

Le bilan complet : propose de **couper** ce que Claude peut déduire du code, et de **déplacer** ce qui ne sert que parfois.

`/insights` : un rapport sur vos sessions passées, pour repérer les erreurs qui reviennent

Intégrés à Claude Code, rien à installer ; un rapport d'abord, rien ne change sans votre accord

Une erreur typique : taper `npm install` dans un monorepo pnpm.
La ligne qui l'évite : « Run scripts with pnpm, not npm » (AGENTS.md de Bénéfriches)

Chaque erreur qui revient devient une règle.

ATELIER 1 Auditer et améliorer votre CLAUDE.md

Phase	Durée	Ce que vous faites
1. Passer au crible	12 min	ni CLAUDE.md ni AGENTS.md ? `/init` d'abord · pour chaque ligne : « si je la retire, Claude fera-t-il une erreur ? » · ajoutez ce que le premier contact a raté
2. Relire en binôme	10 min	avec la grille du livret (slide suivante)
3. Améliorer	15 min	appliquez l'amélioration du binôme (par exemple, une règle rigide reformulée en principe) · extrayez une règle `.claude/rules/` avec `paths` · vérifiez qu'elle se charge
4. Contre-vérifier	8 min	`/doctor prompt-audit` : qu'a-t-il trouvé que vous aviez raté ?

Livrable : votre CLAUDE.md et une règle, commités.

En binôme : chacun travaille sur son repo, le binôme relit. Votre jugement d'abord, l'outil à la fin.

Grille de relecture

Critère	Note	La question
Spécificité	1 à 3	des commandes exactes ? un nouveau pourrait-il s'en servir ?
Bruit	1 à 3	chaque section apporte-t-elle ce que le code ne dit pas ?
Boundaries (limites)	oui / non	les zones interdites sont-elles listées ?
Taille	oui / non	moins de 200 lignes ?

Chaque relecteur donne un point fort et une amélioration concrète.

Restitution express

1

La section la plus **utile** de votre CLAUDE.md

2

Ce que l'**audit** a révélé

3

La **règle** que vous avez extraite, et son `paths`

2 minutes par binôme.

Plan d'adoption 30 jours : ligne 1

Dès lundi, dans le CLAUDE.md de mon équipe, je change

La règle que je propose d'ajouter

J1 matin **ligne 1 : CLAUDE.md et une règle**
Une ligne à la fin de chaque demi-journée · on consolide demain en clôture

Exemple : « j'ajoute les commandes de test ciblées et un golden file ; je propose une règle `angular.md` limitée à `src/app` ».

Si la politique IA de votre entreprise est en cours de définition, ce plan peut la nourrir.

Hooks & skills

CLAUDE.md décrit. Les hooks garantissent. Les skills complètent, à la demande.

Cet après-midi · reprise dans une heure